

First Vibe Coding Exercise

- Build a UGA Snake Game

In class • Any free AI chat tool • nothing to install

The point

Today you become a builder without writing code. You describe what you want, an AI model writes a complete program, you run it, you react, you ask for changes. That loop is called **vibe coding**. The skill is not coding. The skill is describing, evaluating, and steering.

What to build

A playable Snake game in the browser, themed around UGA and Athens, GA. Here is the prompt I used. Copy it, or **write your own**.

STARTER PROMPT

Create a super fun snake game in the theme of UGA and Athens GA. Make it cool, visual, creative. Put everything in a single HTML file I can open in my browser.

The last sentence matters. Asking for one self contained HTML file means you can save it and double click it. No installs, no servers.

Steps

1. **Pick a tool.** ChatGPT, Claude, or Gemini. (Google AI Studio, Lovable also work if you already use them. We'll introduce these tools later in the course too.)
2. **Use the example prompt or use your own prompt.** Wait for the full program. Most tools show a live preview of the game next to the chat (see generative UI below). If your tool only gives you code but not an artifact, copy it into a text editor, save it as `snake.html`, and open the file in Chrome.
3. **Play it.** Does it run? Is it actually themed, or did the model just paint everything red and black? Does it feel like a game someone would show a friend?
4. **Steer it at least one time.** Give feedback in plain language (as if you're a product manager), the way you would to a developer. "The snake is too fast." "Add a scoreboard with Bulldog puns." "Make the food a hedge from Sanford Stadium." "Add sound." "Make it work on my phone." "The game over screen is boring."

5. **Optional: Try a second tool with the same prompt.** Same words, different model, different game. Note what each one did better.

Then submit the html file to eLC -> Snake Game Exercise

What is generative UI

ChatGPT Canvas, Claude Artifacts, and Gemini Canvas all do the same thing: when the model writes something visual, the tool renders it live in a side panel instead of showing raw code. The interface is generated on the fly by the model. This is why vibe coding took off in 2025. The feedback loop went from "copy code, save file, refresh browser" to "look right, it is already running." Each vendor has its own name and its own quirks. The idea is identical.

DO NOT MARRY ONE ECOSYSTEM

The models leapfrog each other every few months. The tool that is best today is not the one that will be best in your first job. Learn the pattern, not the product: describe clearly, demand a runnable result, test it yourself, give specific feedback, repeat. That pattern works in every tool, and it is the same pattern we use in the next exercise to turn a machine learning notebook into a website.

Reflection, with your neighbor

1. Which of your follow up requests worked on the first try, and which needed a second try? What was different about how you phrased them?
2. The model made hundreds of decisions you never specified: colors, speed, controls, scoring. Which ones would you have wanted to control if this were a real product?
3. If a client asked "did you build this?", what is the honest answer?